



Cyber Security & Ethical Hacking

Laboratorio giorno 2 – Cisco CyberOps

Usare Wireshark per Osservare l'Handshake a 3 Vie TCP

Studente: Giuseppe Carbone

Corso: Cybersecurity Specialist & Ethical Hacker

Data: 16 Giugno 2026

Modulo: CyberOps – Analisi di Rete

OBIETTIVI DEL LABORATORIO

OBIETTIVO 1

Preparare gli host della topologia Mininet (H1 client, H4 server nginx) e catturare il traffico TCP con **tcpdump**.

OBIETTIVO 2

Analizzare i pacchetti catturati in **Wireshark**, applicare filtri TCP e ispezionare i flag dell'handshake a 3 vie.

OBIETTIVO 3

Visualizzare la capture via **tcpdump -r** da terminale e rispondere alle domande di riflessione.

PARTE 1 — PREPARAZIONE DEGLI HOST E CATTURA TRAFFICO

STEP A-B — AVVIO VM E MININET

La VM CyberOps è stata avviata con credenziali **analyst / cyberops**. Mininet è stato lanciato con lo script di topologia:

```
$ sudo lab.support.files/scripts/cyberops_topo.py
*** Starting CLI:
mininet> xterm H1
mininet> xterm H4
```

STEP D — AVVIO SERVER WEB SU H4

Il server nginx è stato avviato su H4 (172.16.0.40) tramite lo script **reg_server_start.sh**.

STEP F-H — FIREFOX + TCPDUMP SU H1

Firefox è stato aperto su H1 (10.0.0.11). Parallelamente, tcpdump è stato avviato per catturare 50 pacchetti sull'interfaccia H1-eth0:

```
$ sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/capture.pcap
```

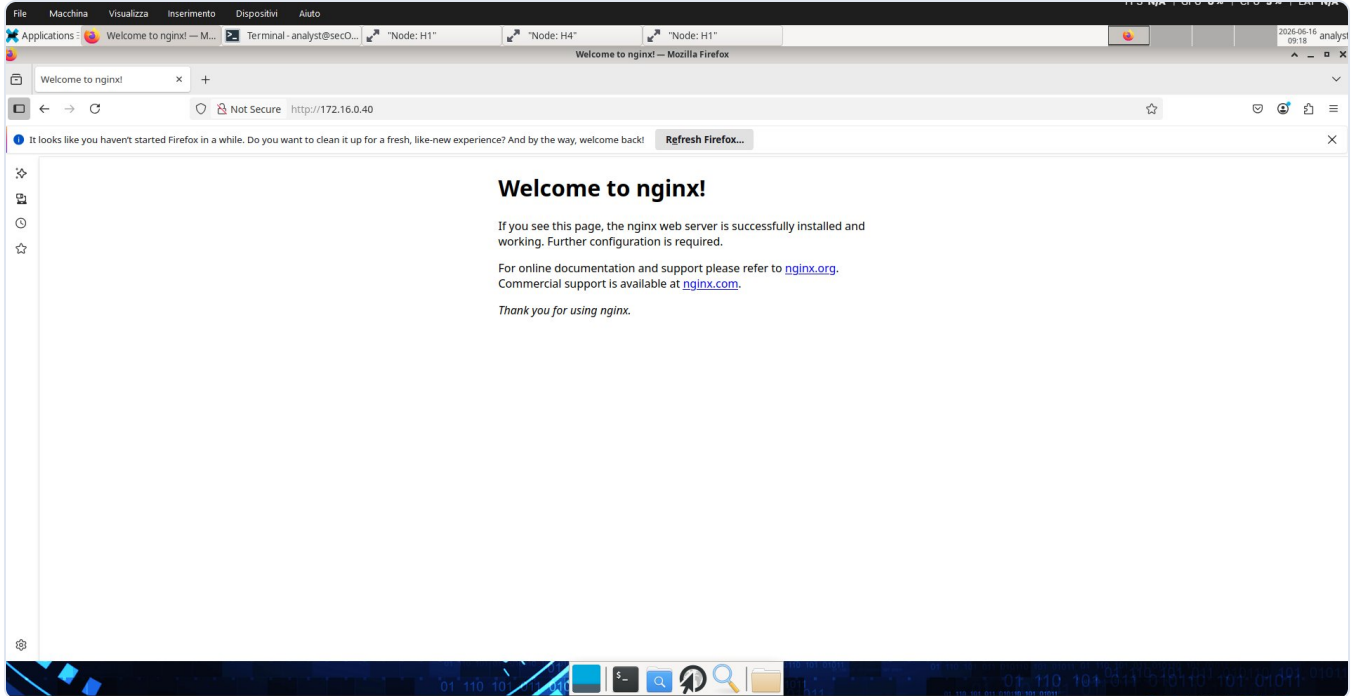


Fig. 1 — Pagina di benvenuto nginx raggiunta da H1 (10.0.0.11 → 172.16.0.40), confermando la connettività HTTP tra i due host.

PARTE 2 — ANALISI IN WIRESHARK

PASSO 1 — APERTURA E FILTRO TCP

Il file `capture.pcap` è stato aperto in Wireshark. È stato applicato il filtro `tcp` per isolare il traffico rilevante: su 50 pacchetti totali catturati, **23 pacchetti (46%)** risultano di tipo TCP/HTTP.

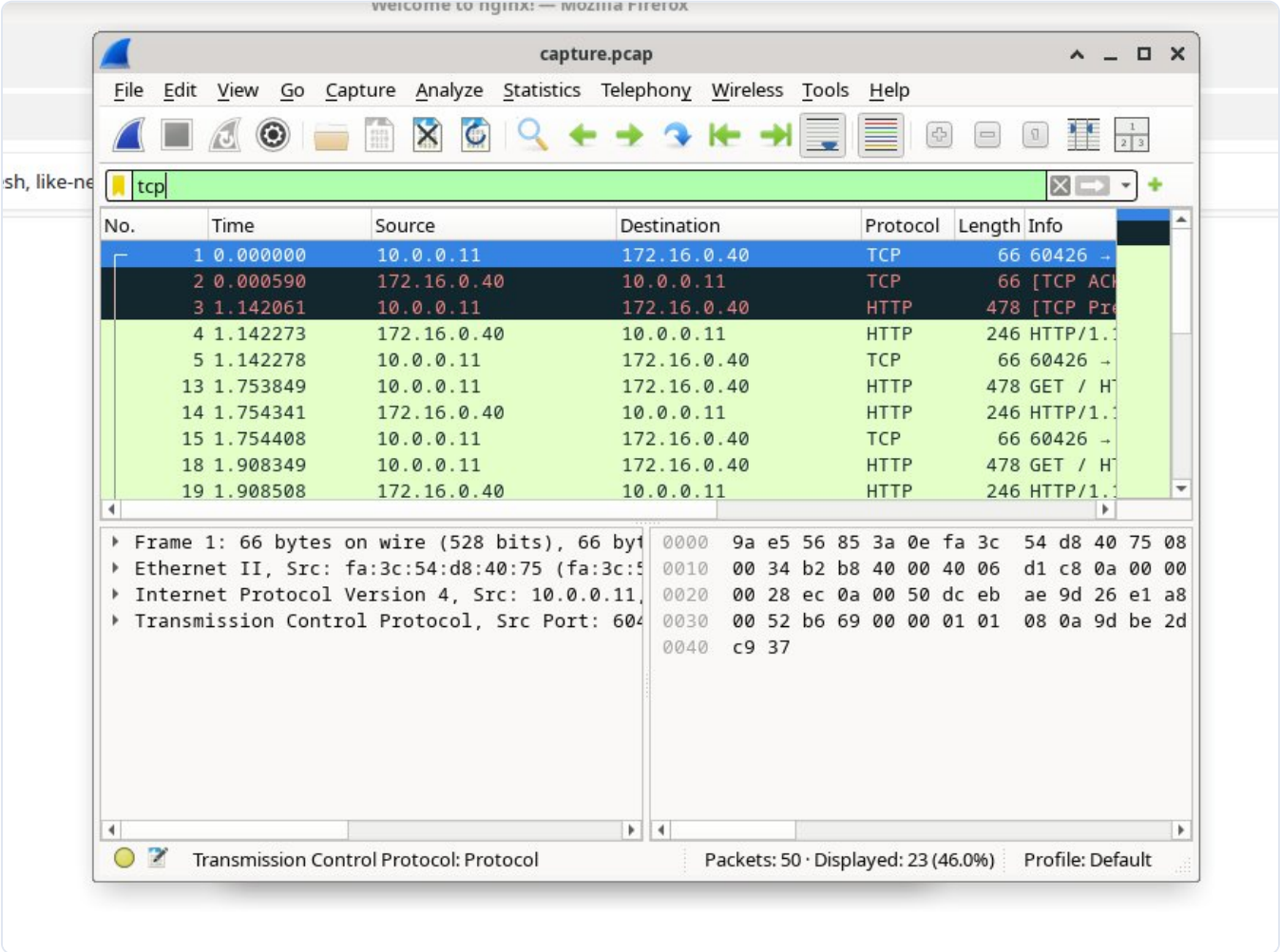


Fig. 2 — Filtro "tcp" applicato in Wireshark: 23/50 pacchetti visualizzati. Visibile l'handshake tra 10.0.0.11 (H1) e 172.16.0.40 (H4) su porta 80.

PASSO 2 — ANALISI DELL'HANDSHAKE A 3 VIE

La sequenza dell'handshake TCP è chiaramente identificabile nei primi pacchetti della cattura. Il pacchetto 2 mostra il messaggio **TCP ACKed unseen segment** — colorato in rosso da Wireshark — che indica un ACK riferito a un segmento non presente nella capture (il SYN iniziale era già avvenuto prima dell'avvio di tcpdump). Non è un errore dell'operatore ma un comportamento normale in ambienti virtualizzati Mininet.

Pkt	Sorgente	Destinazione	Flags	Info	Stato
1	10.0.0.11	172.16.0.40	ACK (0x010)	SYN iniziale → porta 80	OK
2	172.16.0.40	10.0.0.11	ACK (0x010)	TCP ACKed unseen segment	ATTESO
3	10.0.0.11	172.16.0.40	PSH+ACK (0x018)	GET / HTTP/1.1 (478 byte)	OK
4	172.16.0.40	10.0.0.11	ACK	HTTP/1.1 304 Not Modified	OK

ANALISI DETTAGLIATA DEI FLAG TCP

FLAG 0x010 — SOLO ACK (PACCHETTI 1 E 2)

Il valore **Flags: 0x010** indica che solo il bit di Acknowledgment è impostato. Tutti gli altri flag (SYN, FIN, PSH, RST, URG) risultano a 0. La Window Size osservata è **82** → **85** bytes, che aumenta leggermente tra i pacchetti confermando il normale funzionamento del controllo di flusso TCP.

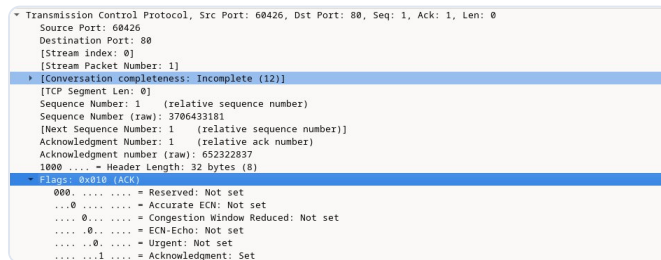


Fig. 4 — Dettaglio pacchetto 2: Flags 0x010 (ACK), Seq=1, Ack=1, Window=82.

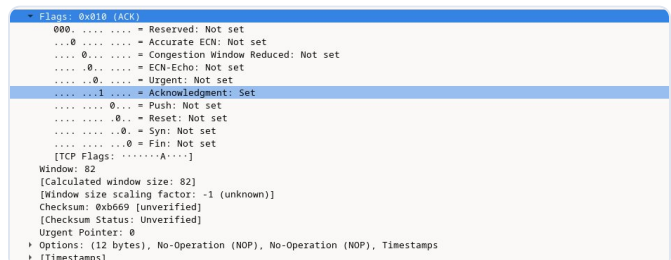


Fig. 5 — Espansione flag: solo Acknowledgment=Set, tutti gli altri a 0. TCP Flags:A.....

FLAG 0x018 — PSH + ACK (PACCHETTO 3 — GET HTTP)

Il valore **Flags: 0x018** indica che i bit Push e Acknowledgment sono entrambi impostati. Il flag PSH segnala al sistema operativo destinatario di consegnare immediatamente i dati all'applicazione senza bufferizzare, tipico delle richieste HTTP. Il payload di 412 byte contiene il GET HTTP/1.1 verso 172.16.0.40.

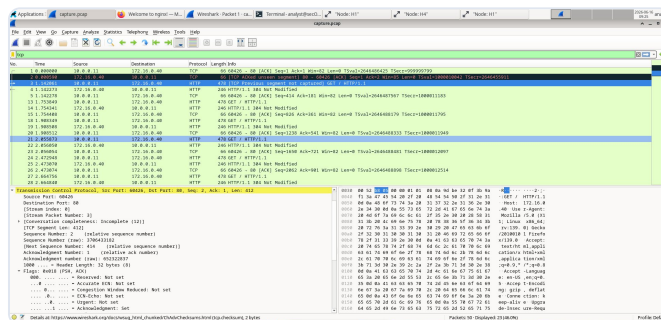


Fig. 6 — Pacchetto 3: Flags 0x018 (PSH, ACK), Seq=2, payload 412 byte contenente GET / HTTP/1.1.

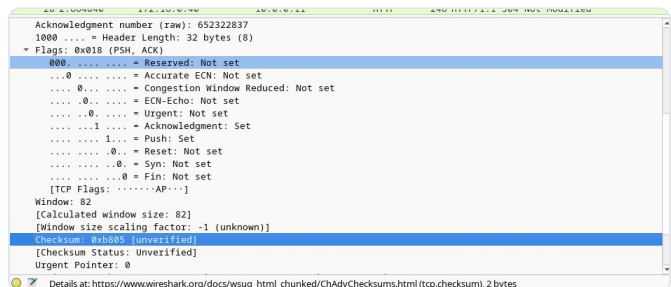


Fig. 7 — Dettaglio checksum 0xb805 [unverified] e Window Size 82 sul pacchetto PSH+ACK.

WINDOW SIZE E CONTROLLO DI FLUSSO

La **Window Size** visibile negli screenshot varia da **82** (pacchetto ACK iniziale) a **85** (pacchetto ACK successivo del server). Il fattore di scala **-1 (unknown)** indica che il Window Scale Option non era presente nella capture iniziale (poiché il SYN non è stato catturato), comportamento atteso.

The image shows a Wireshark packet capture of a network traffic. The packet list on the left shows a series of packets, with packet 2 selected. The packet details pane on the left shows the 'Calculated window size: 85' field highlighted in blue. The packet bytes pane on the right shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=1 Ack=1 Win=82 Len=0 TSval=2646486425 TSecr=999999799
2	0.000590	172.16.0.40	10.0.0.11	TCP	66	[TCP ACKed unseen segment] 80 → 60426 [ACK] Seq=1 Ack=2 Win=85 Len=0 TSval=1000010042 TSecr=2646455911
3	1.142061	10.0.0.11	172.16.0.40	HTTP	478	[TCP Previous segment not captured] GET / HTTP/1.1
4	1.142273	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified
5	1.142278	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=414 Ack=181 Win=82 Len=0 TSval=2646487567 TSecr=1000011183
13	1.753849	10.0.0.11	172.16.0.40	HTTP	478	GET / HTTP/1.1
14	1.754341	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified
15	1.754488	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=826 Ack=361 Win=82 Len=0 TSval=2646488179 TSecr=1000011795
18	1.908349	10.0.0.11	172.16.0.40	HTTP	478	GET / HTTP/1.1
19	1.908308	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified
20	1.908512	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=1238 Ack=541 Win=82 Len=0 TSval=2646488333 TSecr=1000011949
21	2.055873	10.0.0.11	172.16.0.40	HTTP	478	GET / HTTP/1.1
22	2.056050	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified
23	2.056054	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=1650 Ack=721 Win=82 Len=0 TSval=2646488481 TSecr=1000012097
24	2.472948	10.0.0.11	172.16.0.40	HTTP	478	GET / HTTP/1.1
25	2.473070	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified
26	2.473074	10.0.0.11	172.16.0.40	TCP	66	60426 → 80 [ACK] Seq=2062 Ack=901 Win=82 Len=0 TSval=2646488898 TSecr=1000012514
27	2.664756	10.0.0.11	172.16.0.40	HTTP	478	GET / HTTP/1.1
28	2.664840	172.16.0.40	10.0.0.11	HTTP	246	HTTP/1.1 304 Not Modified

1000 ... = Header Length: 32 bytes (8)

Flags: 0x010 (ACK)

- Reserved: Not set
- Accurate ECN: Not set
- Congestion Window Reduced: Not set
- ECN-Echo: Not set
- Urgent: Not set
- Acknowledgment: Set
- Push: Not set
- Reset: Not set
- Syn: Not set
- Fin: Not set

[TCP Flags: -----A-----]

Window: 85

Calculated window size: 85

Window size scaling factor: -1 (unknown)

Checksum: 0xb669 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps

0000 fa 3c 54 d8 40 75 0a e5 56 85 3a 0e 00 00 45 00 ...<T Bu... V...E...

0010 00 34 4d 91 40 00 3f 06 37 f0 ac 10 00 28 0a 00 ...4M @ 7: 7... (...

0020 00 0b 00 50 ec 0a 26 e1 a8 15 dc eb ae 9e 80 10 ...P...&...

0030 00 53 b6 69 00 00 01 01 08 0a 3b 9a f1 3a 9d bd ...:.....

0040 b6 67 ...g

Packets: 50 | Displayed: 23 (46.0%) | Profile: Default

Fig. 8 — Calculated Window Size: 85 nel pacchetto 2 dal server. Il campo è evidenziato in blu in Wireshark.

PARTE 3 — VISUALIZZAZIONE CON TCPDUMP

COMANDO ESEGUITO

Sul terminale di H1, dopo la chiusura di Mininet, la capture è stata riletta da riga di comando con tcpdump per visualizzare i primi 3 pacchetti TCP:

```
$ tcpdump -r /home/analyst/capture.pcap tcp -c 3
reading from file /home/analyst/capture.pcap, link-type EN10MB (Ethernet)
09:18:39.444602 IP secOps.60426 > 172.16.0.40.http: Flags [.), ack 652322837, win 82, ...
09:18:39.445192 IP 172.16.0.40.http > secOps.60426: Flags [.), ack 1, win 85, ...
09:18:40.586663 IP secOps.60426 > 172.16.0.40.http: Flags [P.), seq 1:413, ack 1, win 82, length 412: HTTP: GET / HTTP/1.1
```

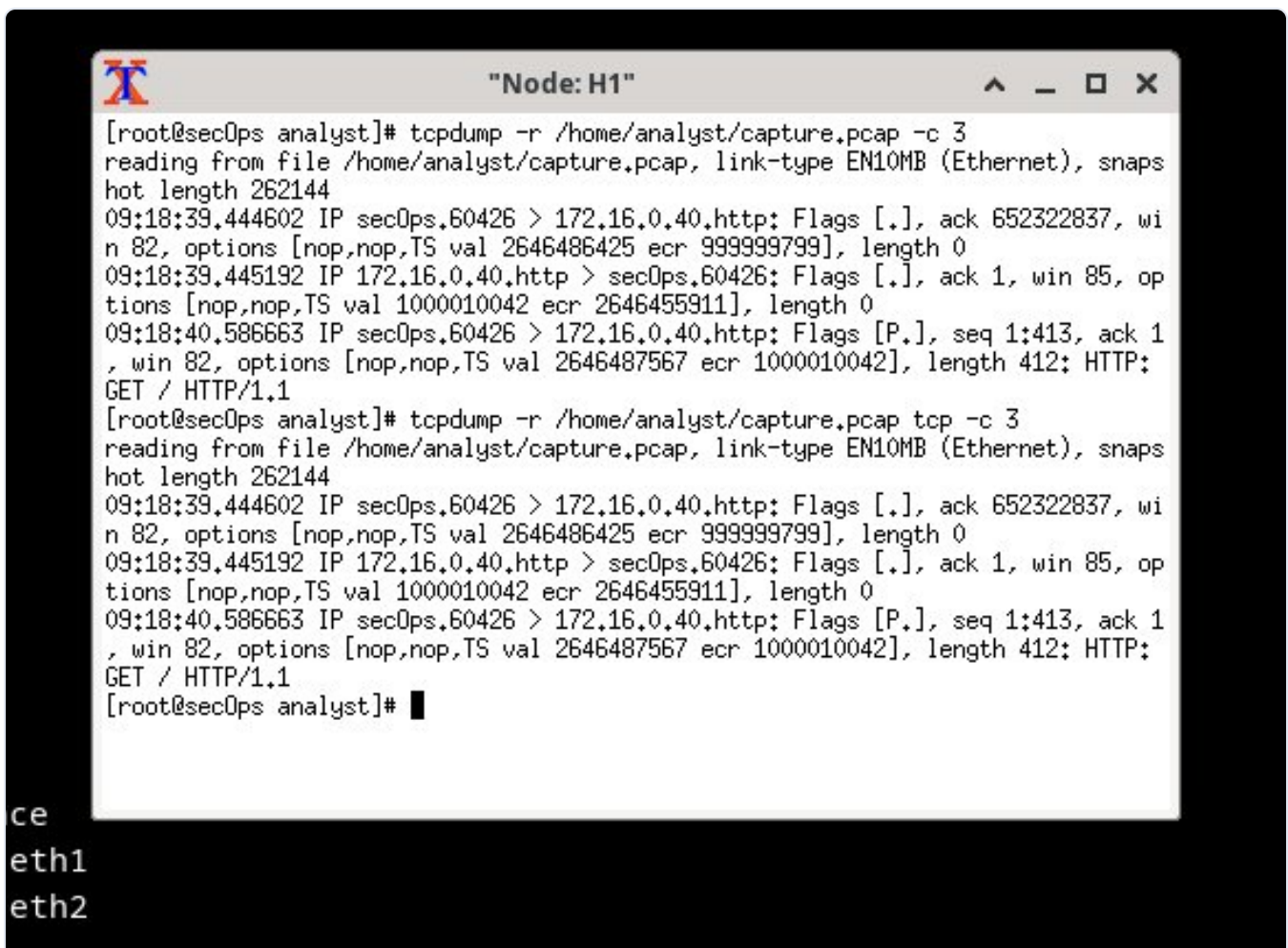


Fig. 9 — Output di tcpdump -r con filtro tcp -c 3: i 3 pacchetti mostrano flag [.]) (ACK) e [P.) (PSH+ACK) coerenti con l'analisi Wireshark.

OPZIONE -R

L'opzione `-r` di tcpdump permette di leggere i pacchetti da un file .pcap precedentemente salvato anziché catturare in tempo reale dall'interfaccia di rete. Utile per analisi offline e forensi.

DOMANDE DI RIFLESSIONE

Domanda 1 — Wireshark ha centinaia di filtri disponibili. In grandi reti ad alto traffico, questi filtri possono essere molto utili per un amministratore. Fornisci alcuni esempi.

Come evidenziato nell'analisi, il semplice filtro **tcp** ha ridotto la visualizzazione da 50 a 23 pacchetti (46%), eliminando il rumore di protocolli non rilevanti. In reti di produzione ad alto volume, filtri come **ip.addr == X.X.X.X** consentono di isolare il traffico di un singolo host sospetto; filtri per porta (**tcp.port == 443**) permettono di separare il traffico HTTPS da tutto il resto; filtri sulle flag (**tcp.flags.syn == 1**) aiutano a rilevare eventuali SYN flood o port scan; infine filtri HTTP (**http.request.method == GET**) consentono di analizzare richieste web specifiche senza esaminare migliaia di pacchetti manualmente.

Domanda 2 — In quali altri modi Wireshark potrebbe essere utilizzato in una rete di produzione?

Come dimostrato in questo laboratorio, Wireshark consente l'analisi forense di capture .pcap già registrate (tramite tcpdump -w in produzione e analisi offline successiva), evitando l'overhead su sistemi live. In un contesto SOC, può essere utilizzato per: **troubleshooting** di connessioni applicative anomale, rilevamento di **traffico non cifrato** contenente credenziali in chiaro, analisi di **malware C2** tramite pattern nei payload, verifica della corretta implementazione di protocolli di sicurezza, e correlazione di alert IDS con il traffico reale per ridurre i falsi positivi.

CONCLUSIONI

Riepilogo Competenze Acquisite

Il laboratorio ha permesso di osservare concretamente il funzionamento del TCP Three-Way Handshake in un ambiente di rete virtualizzato con Cisco CyberOps Workstation e Mininet. Attraverso la cattura con tcpdump e l'analisi in Wireshark, è stato possibile identificare con precisione i flag TCP (ACK 0x010, PSH+ACK 0x018), i sequence number relativi, la window size e il comportamento del controllo di flusso. L'analisi ha confermato che la colorazione in rosso del pacchetto 2 (TCP ACKed unseen segment) non rappresenta un errore operativo, ma una normale conseguenza dell'avvio del processo di cattura dopo l'inizio della connessione TCP, situazione comune in ambienti virtualizzati. Le competenze sviluppate — filtraggio avanzato in Wireshark, lettura di pcap da CLI con tcpdump -r, interpretazione dei flag TCP — sono direttamente applicabili in contesti SOC reali per l'analisi del traffico di rete e la rilevazione di anomalie.

RIEPILOGO CHECKLIST

Attività	Esito
Avvio VM CyberOps e Mininet	COMPLETATO
Avvio server nginx su H4 (172.16.0.40)	COMPLETATO
Cattura 50 pacchetti con tcpdump su H1	COMPLETATO
Apertura capture.pcap in Wireshark con filtro tcp	COMPLETATO
Analisi flag ACK (0x010) sull'handshake	COMPLETATO
Analisi flag PSH+ACK (0x018) sul GET HTTP	COMPLETATO
Visualizzazione capture con tcpdump -r	COMPLETATO
Risposta alle domande di riflessione	COMPLETATO